

Bank Management System

Java • Spring Framework • MySQL • Maven

Console-Based Banking Application with DAO Architecture & Transaction Management

PART 1

GitHub Repository Description

Overview

A console-based Bank Management System built with Core Java, Spring Framework (IoC & JDBC), and MySQL. The project demonstrates real-world layered architecture, programmatic transaction management, custom exception handling, and database trigger automation — simulating essential banking operations through a menu-driven interface.

Features

#	Feature	Description
1	Create Account	Register a new customer with full field-level validation (account number, name, mobile, Aadhaar, address, account type, opening balance)
2	Deposit Amount	Deposit funds into an existing account with automatic balance update
3	Withdrawal Amount	Withdraw funds with real-time balance sufficiency check before processing
4	Fund Transfer	Atomic debit-credit transfer between two accounts using Spring TransactionTemplate — all-or-nothing execution
5	View Account Details	Retrieve complete customer profile and current balance by account number
6	View Deposit Details	Fetch deposit records (deposit ID, amount) for a given account
7	View Withdrawal Details	Fetch withdrawal records for a given account
8	View Transaction Details	Fetch full transaction history including sender, receiver, type, and amount

Tech Stack

Layer	Technology
Language	Java (Core Java, Java 8+)
Framework	Spring Framework 5.x (Spring Core, Spring JDBC)
Database	MySQL 8.x
DB Access Layer	Spring JdbcTemplate + RowMapper implementations
Transaction Management	Spring TransactionTemplate (Programmatic)
Dependency Injection	XML-based Spring IoC (bank.xml)
Build Tool	Maven (pom.xml)
Exception Handling	Custom Checked Exception Hierarchy

Project Structure

```

src/
  main/
    java/com/bankmanagement/
      TestBankManagement.java      ← Main entry point (menu loop)
    entities/
      CreateAccount.java          ← Account entity + console input logic
      Deposits.java               ← Deposit entity + input logic
      Withdrawals.java            ← Withdrawal entity + input logic
      TransactionHistory.java      ← Transaction entity + input logic
      AccountView.java            ← View input for account lookup
      DepositsView.java           ← View input for deposit lookup
      WithdrawalView.java         ← View input for withdrawal lookup
      TransactionView.java        ← View input for transaction lookup
    dao/
      BankDao.java                ← DAO Interface (contract)
      BankDaoImpl.java            ← DAO Implementation (all DB logic)
      AccountMapperImpl.java      ← RowMapper: ResultSet -> CreateAccount
      DepositMapperImpl.java      ← RowMapper: ResultSet -> Deposits
      WithdrawalMapperImpl.java   ← RowMapper: ResultSet -> Withdrawals
      TransactionMapperImpl.java  ← RowMapper: ResultSet -> TransactionHistory
    customexception/
      InvalidAccountNumberException.java
      InvalidMobileNumberException.java
      InvalidDepositIdException.java
      + 4 more custom exceptions
    resources/
      bank.xml                    ← Spring IoC XML configuration
  sql/
    bankmanagementsystems.sql     ← DB schema + triggers + setup

```

Database Schema

Database: **bankmanagement_db**

Table	Primary Key	Auto-ID Format	Description
createaccount	account_number (BIGINT)	—	Customer profile + live balance
deposits	deposit_id (VARCHAR)	D00001, D00002 ...	Deposit records (FK → createaccount)
withdrawals	withdrawal_id (VARCHAR)	W00001, W00002 ...	Withdrawal records (FK → createaccount)
transactionhistory	transaction_id (VARCHAR)	T00001, T00002 ...	Fund transfer records (FK → createaccount)

All child tables use *ON DELETE CASCADE* — deleting an account automatically removes all associated deposit, withdrawal, and transaction records.

Primary IDs for deposits, withdrawals, and transactions are auto-generated by MySQL *BEFORE INSERT* triggers, keeping ID management entirely out of the application layer.

Custom Exception Handling

All user inputs are validated using a hierarchy of custom checked exceptions, providing precise and user-friendly error messages at every input step:

Exception Class	Validates
InvalidAccountNumberException	Enforces exactly 14-digit account numbers
InvalidMobileNumberException	Enforces 10-digit numeric-only mobile numbers
InvalidDepositIdException	Validates deposit ID format
InvalidAadhaarNumberException	Enforces exactly 12-digit Aadhaar numbers
InvalidAccountHolderNameException	Rejects names containing digits or special characters
InvalidAddressException	Alphanumeric-only address validation
InvalidAmountNegativeException	Prevents negative balance or amount inputs

Spring Configuration (bank.xml)

- DataSource bean — MySQL JDBC connection (url, username, password, driver)
- JdbcTemplate bean — wired with the DataSource for all SQL operations
- BankDaoImpl bean — injected with JdbcTemplate and PlatformTransactionManager via setter injection
- DataSourceTransactionManager bean — manages programmatic transactions for atomic fund transfers

How to Run

Prerequisites

- Java 8 or higher
- MySQL 8.x
- Maven 3.x

Steps

```
# 1. Clone the repository
git clone https://github.com/your-username/bank-management-system.git

# 2. Set up the database
mysql -u root -p < bankmanagementsystems.sql

# 3. Update DB credentials in src/main/resources/bank.xml
#   Set: url, username, password

# 4. Build the project
mvn clean install

# 5. Run the application
mvn exec:java -Dexec.mainClass="com.bankmanagement.TestBankManagement"
```

Key Concepts Demonstrated

- Spring IoC & Dependency Injection via XML configuration (bank.xml)
- Spring JdbcTemplate for clean, boilerplate-free database operations
- Programmatic Transaction Management using TransactionTemplate
 - Fund transfer executes debit + credit + history insert atomically
 - Any failure triggers automatic rollback via status.setRollbackOnly()
- RowMapper Pattern — clean separation of ResultSet mapping into dedicated Impl classes

- Custom Exception Hierarchy — 7+ exception classes for granular input validation
- MySQL BEFORE INSERT Triggers — auto-generate formatted primary IDs (D/W/T prefix)
- Enum-based Validation — account types (saving/current) and transaction types (online/cash)
- Interface-driven DAO Design — BankDao interface decouples contract from implementation

PART 2

LinkedIn Project Description

https://www.linkedin.com/posts/mohit-kumar-07m_java-springframework-springjdbc-ugcPost-7470579425068097536-r9n5/?utm_source=share&utm_medium=member_desktop&rcm=ACoAAGlr0qMBth8rN2Eion9uSTucfEvzL2uQKBU

Title

Bank Management System | Java • Spring Framework • MySQL

Description

Built a fully functional console-based Bank Management System using Core Java, Spring Framework (IoC & JDBC), and MySQL — implementing real-world banking operations with clean architectural patterns and production-style code organization.

What the Project Does

Enables customers to create bank accounts, deposit and withdraw funds, transfer money between accounts, and view complete transaction records — all from a menu-driven console interface with thorough input validation at every step.

Technical Highlights

- Designed a layered DAO architecture using the BankDao interface and BankDaoImpl, keeping database logic cleanly separated from the main application flow
- Used Spring JdbcTemplate with four custom RowMapper implementations (AccountMapperImpl, DepositMapperImpl, WithdrawalMapperImpl, TransactionMapperImpl) to map SQL result sets into Java objects without boilerplate code

- Implemented atomic fund transfers using Spring's TransactionTemplate — ensuring debit, credit, and transaction history insert either all succeed or all roll back together, preventing partial transfers
- Configured the entire application with XML-based Spring IoC (bank.xml), wiring DataSource, JdbcTemplate, PlatformTransactionManager, and DAO beans through setter injection
- Wrote MySQL BEFORE INSERT triggers that auto-generate formatted primary IDs (D00001, W00001, T00001) on every insert — removing all ID management from the application layer
- Built a custom exception hierarchy with 7+ exception classes covering invalid account numbers, mobile numbers, Aadhaar numbers, addresses, and negative amounts — providing precise, user-friendly error messages at every console input step
- Modelled all entities (CreateAccount, Deposits, Withdrawals, TransactionHistory) with both data fields and static input-collection methods, keeping the main TestBankManagement class clean and readable

Tech Stack

Java • Spring Core • Spring JDBC • JdbcTemplate • TransactionTemplate • MySQL 8.x • Maven • SQL Triggers • RowMapper • Custom Exceptions • XML-based DI

What I Learned

This project strengthened my understanding of Spring's dependency injection model, programmatic transaction management for atomic multi-step database operations, and how to build a production-style DAO layer with interface-driven design from scratch.

GitHub Link: <https://github.com/Mohitsrivastava07/Java-Internship-Projects/tree/main/Task02-BankManagementSystem>

☐ Bug Note (Fix Before Publishing)

In TestBankManagement.java, choices 7 and 8 incorrectly use depositsView.getAccountNumber() instead of withdrawalView.getAccountNumber() and transactionView.getAccountNumber() respectively. This causes a NullPointerException if choices 7 or 8 are selected without first going through choice 6.